

CS F364: Design & Analysis of Algorithms

Quiz 1 Solution – June 2, 2026

Part (a) — Recurrence

The algorithm:

- **Divides** an array of size n into 3 parts of size $n/3$ each $\rightarrow O(1)$ or $\Theta(1)$.
- **Conquers** by recursively processing 2 of the 3 parts $\rightarrow 2T(n/3)$.
- **Combines** by scanning the entire array $\rightarrow \Theta(n)$.

Hence the recurrence is:

$$T(n) = 2T(n/3) + \Theta(n), \quad T(1) = \Theta(1)$$

Part (b) — Recursion Tree Method

Tree structure:

- At the root (level 0), the problem size is n , cost = cn (for some constant c).
- The root spawns 2 children, each of size $n/3$. Cost per child = $c \cdot n/3$.
- At level k , each node has size $n/3^k$, and there are 2^k such nodes.
- The tree bottoms out when $n/3^k = 1$, i.e., $k = \log_3 n$.

Cost per level:

- Level k : $2^k \cdot c \cdot \frac{n}{3^k} = cn \left(\frac{2}{3}\right)^k$

Number of levels: $\log_3 n + 1$ (levels 0 through $\log_3 n$).

Total cost:

$$\begin{aligned} T(n) &= \sum_{k=0}^{\log_3 n} cn \left(\frac{2}{3}\right)^k \\ &= cn \sum_{k=0}^{\log_3 n} \left(\frac{2}{3}\right)^k \end{aligned}$$

This is a geometric series with ratio $r = 2/3 < 1$. Even if we extend the series to infinity, it converges:

$$\sum_{k=0}^{\infty} \left(\frac{2}{3}\right)^k = \frac{1}{1 - 2/3} = 3$$

Therefore:

$$T(n) \leq 3cn = \Theta(n)$$

Part (c) — Final Bound

$$T(n) = \Theta(n)$$

The linear combine step dominates; the recursive work shrinks geometrically and sums to a constant factor of n .

Bonus

If the algorithm processed **all three** parts, the recurrence becomes:

$$T(n) = 3T(n/3) + \Theta(n)$$

At level k : $3^k \cdot c \cdot \frac{n}{3^k} = cn$, and there are $\log_3 n + 1$ levels, so:

$$T(n) = \sum_{k=0}^{\log_3 n} cn = cn \cdot (\log_3 n + 1) = \Theta(n \log n)$$

$$T(n) = \Theta(n \log n)$$